BLM0462 – Örüntü Tanıma

(Pattern Recognition)

Ünite 4

Parametrik Olmayan Yöntemler

(Non-Parametric Methods)

Prof. Dr. Turgay Tugay BİLGİN

Bilgisayar Mühendisliği Bölümü

3. Sınıf Ders Notu

Kaynak: Duda, Hart & Stork, *Pattern Classification*, Bölüm 4

1 k-En Yakın Komşular (k-NN) ile Yoğunluk Kestirimi

Yoğunluk kestirimi için kullanılan temel yaklaşım yine aynıdır:

$$p(x) \approx \frac{k/n}{V} \tag{1}$$

Ancak burada kritik bir fark vardır. Parzen pencereleri ile k-NN yöntemlerinin bu formülü kullanış biçimleri birbirinden farklıdır.

1.1 Parzen ve k-NN Arasındaki Fark

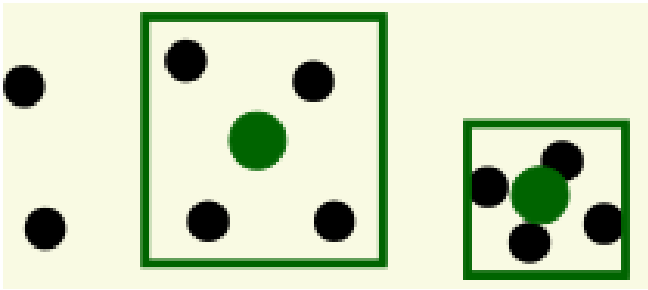
Parzen Pencereleri	**k-NN**
V sabit, k değişiyor	k sabit, V değişiyor

Parzen yönteminde pencere boyutu V önceden belirlenir ve bu pencereye düşen veri sayısı k her noktada farklı çıkar. k-NN yönteminde ise tam tersi geçerlidir: komşu sayısı k önceden sabitlenir ve bu sayıda komşuyu içine alacak bölgenin hacmi V her noktada farklı olur.

1.2 k-NN Nasıl Çalışır?

Yöntem şu adımlarla uygulanır:

1. x noktasını al.
2. Etrafındaki bölgeyi büyütmeye başla.
3. İçine k tane veri noktası girene kadar bölgeyi genişlet.
4. O anki bölgenin hacmi V olarak belirlenir.



Şekil 1: k-NN yoğunluk kestirimi. Yeşil kutular x etrafında oluşturulan bölgeleri göstermektedir. Seyrek bölgede kutu büyür (V büyük), yoğun bölgede ise kutu küçük kalır (V küçük).

Şekil 1'deki görselde bu durum açıkça görülmektedir. Yeşil kutular x etrafında oluşturulan bölgeleri temsil eder. Verinin seyrek olduğu bölgede k komşuya ulaşmak için kutu büyümek zorundadır; verinin yoğun olduğu bölgede ise az bir genişleme ile k komşu zaten yakalanır ve kutu küçük kalır.

1.3 Kritik Avantaj

k-NN yönteminin en önemli avantajı, Parzen yöntemindeki gibi bir pencere genişliği h seçmek zorunda olunmamasıdır. Model, verinin yoğunluğuna göre kendini otomatik olarak adapte eder.

Sonuç

k-NN yoğunluk kestiriminde komşu sayısı k sabit tutulur, pencere boyutu V ise değişkendir. Yoğun bölgelerde V küçük olduğu için yüksek yoğunluk, seyrek bölgelerde V büyük olduğu için düşük yoğunluk elde edilir. Kısaca: k-NN = sabit sayıda komşu, değişken pencere.

2 k-En Yakın Komşu ile Yoğunluk Kestirimi: Pratik Sınırlamalar

k-NN yaklaşımı ile doğrudan $p(x)$ yoğunluğunu kestirmek pratikte iyi çalışmaz. Bunun temel nedenleri şunlardır:

- Elde edilen sonuç her zaman geçerli bir yoğunluk fonksiyonu değildir.
- Yoğunluk fonksiyonu çok sayıda süreksizlik içerir; sivri ve türevlenemez (non-differentiable) bir yapıdadır.
- Gözlem olmayan geniş bölgelerde bile yoğunluk sıfıra yaklaşmaz; kuyruklar gereğinden ağır olur (heavy tails).

Teorik olarak, örnek sayısı sonsuza gittiğinde k-NN ile gerçek yoğunluğa yakınsayan tahminler elde edilebilir. Ancak pratikte veri sayısı her zaman sınırlıdır ve bu nedenle bu sonuç çoğu durumda kullanışlı değildir.

Önemli

k-NN yöntemi doğrudan yoğunluk kestirimi için uygun olmasa da, ileride görüleceği gibi **sınıflandırma** (classification) problemlerinde oldukça etkili bir araç olarak kullanılmaktadır.

3 k-En Yakın Komşu (k-NN) ile Sınıflandırma

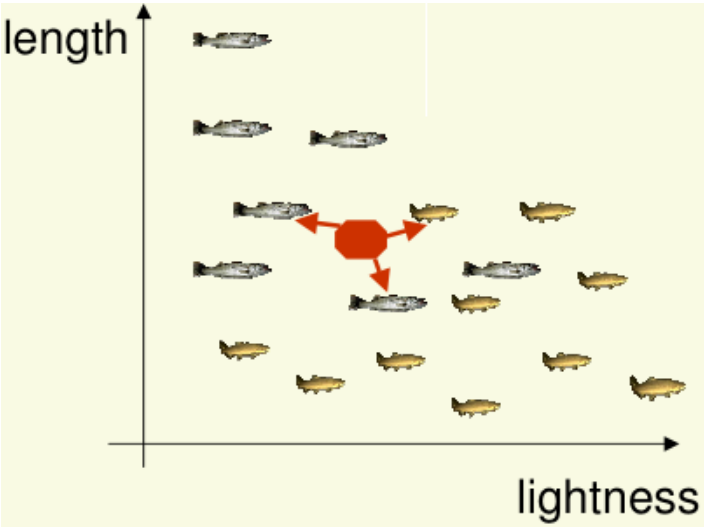
k-NN'in temel yorumu oldukça basittir. Etiketsiz bir örnek x verildiğinde, veri kümesindeki en benzer k adet örnek bulunur ve bu komşular arasında en sık görülen sınıf x'e atanır.

3.1 Nasıl Çalışır?

Verilen bir x noktası için yöntem şu adımlarla uygulanır:

1. En yakın k komşu bulunur.
2. Bu komşuların sınıf etiketlerine bakılır.
3. En çok tekrar eden sınıf seçilir (çoğunluk oylaması).

3.2 Örnek: Balık Sınıflandırma



Şekil 2: k-NN sınıflandırma örneği. Yatay eksen açıklık (lightness), dikey eksen uzunluk (length) özelliğini göstermektedir. Farklı balık türleri (levrek ve somon) özellik uzayında dağılmıştır. Kırmızı nokta sınıfı bilinmeyen yeni örnektir; oklar en yakın $k = 3$ komşuyu işaret etmektedir.

Görselde iki özellik kullanılmaktadır: yatay eksende açıklık (lightness), dikey eksende uzunluk (length). Noktalar farklı balık türlerini (levrek ve somon) temsil etmektedir. Kırmızı nokta ise sınıfı bilinmeyen yeni bir örnektir.

$k = 3$ seçildiğinde karar şu şekilde verilir:

- Kırmızı noktaya en yakın 3 komşuya bakılır.
- Bu komşulardan 2 tanesi levrek (sea bass), 1 tanesi somondur.
- Çoğunluk levrek olduğu için yeni örnek **levrek** olarak sınıflandırılır.

Sonuç

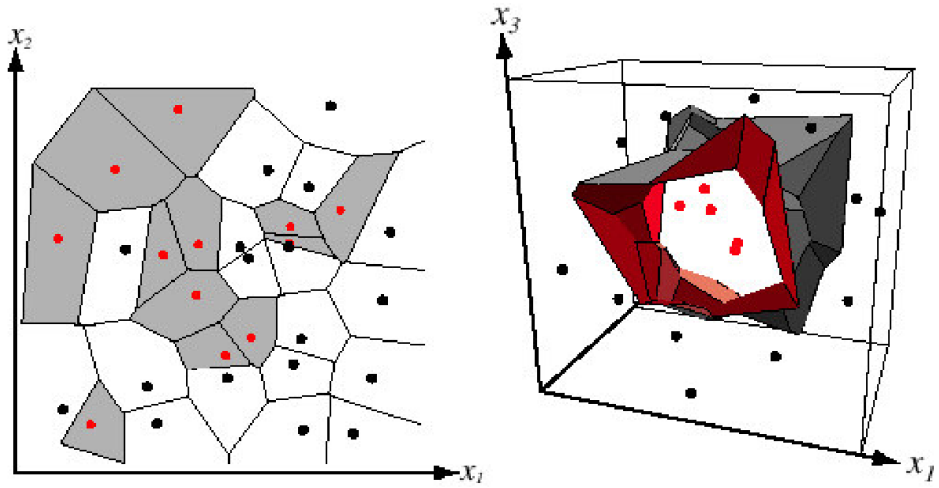
k-NN sınıflandırması benzerlik temelli çalışır, parametrik bir model kurmaz ve kararı tamamen komşuların çoğunluğuna göre verir. Kısaca: "En yakınlarına bak, çoğunluk kimse o sınıfı ver."

4 1-NN ve Voronoi Hücreleri

k-NN'i daha iyi anlamanın en net yolu, $k = 1$ durumuna bakmaktır. Bu durumda modelin yaptığı şey çok basittir: her yeni nokta x, en yakın tek veri noktasının sınıfını alır. Bu basit kural aslında veri uzayını otomatik olarak parçalara ayırır. Bu parçalara **Voronoi hücreleri** (Voronoi cells) denir.

4.1 Temel Fikir

Her eğitim noktası x_i, kendi etki alanına sahiptir. Bu alan, uzaydaki tüm noktalar içinde x_i'ye en yakın olanların kümesidir. Sonuç olarak uzay tamamen bölünür ve her bölge bir veri noktasına ait olur.



Şekil 3: Voronoi hücreleri. Sol: 2 boyutlu uzayda düzlem küçük çokgenlere bölünmüştür; her çokgen içindeki tek bir veri noktasına aittir. Sağ: Aynı mantık 3 boyutta uygulanmış; hücreler çok yüzlü (polyhedron) yapılardır. Renkler farklı sınıfları, sınırlar ise iki noktaya eşit uzaklıktaki yerleri göstermektedir.

4.2 2 Boyutlu Durum (Soldaki Şekil)

Şekil 3'nin sol tarafında düzlem küçük çokgenlere bölünmüştür. Her çokgen içindeki tek bir veri noktasına aittir ve renkler farklı sınıfları göstermektedir. Çokgenler arasındaki sınırlar, iki noktaya eşit uzaklıktaki yerlerdir. Bu sınırlar aynı zamanda **karar sınırlarıdır**; k-NN sınıflandırması bu bölgelerle yapılır.

4.3 3 Boyutlu Durum (Sağdaki Şekil)

Şekil 3'nin sağ tarafında aynı mantık 3 boyutta uygulanmıştır. Hücreler artık çok yüzlü (polyhedron) yapılardır ve kristal benzeri parçalı bir görüntü oluşur. Boyut arttıkça yapı daha karmaşık hale gelir.

4.4 k-NN ile Bağlantı

$k = 1$ durumunda Voronoi hücreleri net ve keskin sınırlar üretir. $k > 1$ olduğunda ise çoğunluk oylaması devreye girer ve bu sınırlar yumuşar.

Önemli

1-NN aslında uzayı "en yakın noktaya göre" bölmektedir. Bu nedenle karar sınırları çok parçalıdır ve veri dağılımına çok duyarlıdır.

Sonuç

Voronoi diyagramı, 1-NN'in geometrik yorumudur ve k-NN'in nasıl karar verdiğini görselleştirmenin en güçlü yoludur. Kısaca: 1-NN = uzayı en yakın veri noktasına göre parçalara bölmek.

5 1-NN ve Voronoi Diyagramı: Python Uygulaması

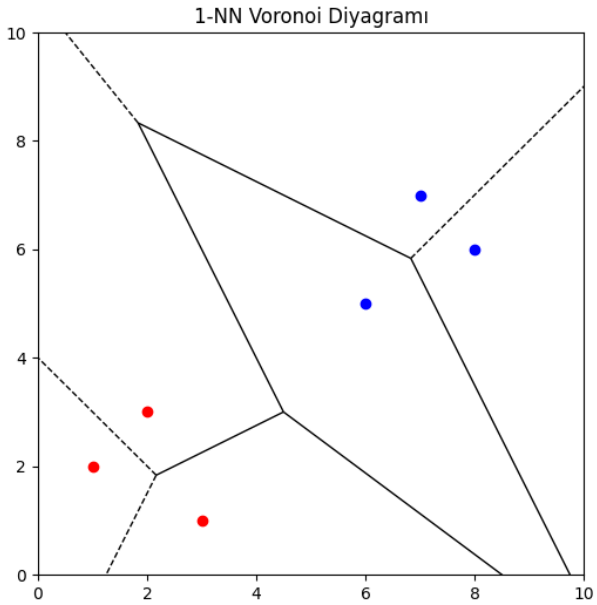
1-en yakın komşu (1-NN) yönteminde, her nokta kendisine en yakın eğitim örneğinin sınıfını alır. Bu durum, veri uzayının otomatik olarak bölünmesine yol açar. Bu bölünme **Voronoi diyagramı** ile ifade edilir. Her hücre, içindeki tüm noktaların en yakın olduğu veri örneğine karşılık gelir.

Aşağıdaki örnekte, iki sınıfa ait basit bir 2 boyutlu veri kümesi kullanılarak Voronoi diyagramı oluşturulmuştur. Böylece 1-NN'in karar sınırları doğrudan görselleştirilir.

5.1 Python ile Voronoi Diyagramı

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import Voronoi, voronoi_plot_2d

# Veri noktaları (2 sınıf)
points = np.array([
[1, 2], [2, 3], [3, 1],   # sınıf 0
[6, 5], [7, 7], [8, 6]    # sınıf 1
])
labels = np.array([0, 0, 0, 1, 1, 1])

# Voronoi diyagramı oluştur
vor = Voronoi(points)

# Çizim
```

```
fig, ax = plt.subplots(figsize=(6, 6))
voronoi_plot_2d(vor, ax=ax, show_vertices=False,
line_colors='black')

# Noktaları sınıflarına göre göster
for i, point in enumerate(points):
if labels[i] == 0:
ax.plot(point[0], point[1], 'ro')  # kırmızı
else:
ax.plot(point[0], point[1], 'bo')  # mavi

ax.set_title("1-NN Voronoi Diyagramı")
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
plt.show()
```



Şekil 4: Python ile oluşturulan 1-NN Voronoi diyagramı. Kırmızı noktalar sınıf 0'ı, mavi noktalar sınıf 1'i temsil etmektedir. Düz çizgiler sonlu Voronoi kenarlarını, kesikli çizgiler ise sonsuza uzanan (sınırsız) Voronoi kenarlarını göstermektedir. Her hücre bir veri noktasına aittir ve hücre sınırları 1-NN karar sınırlarını temsil eder.

5.2 Açıklama

Kodda kullanılan temel bileşenler şunlardır:

- `points`: Veri uzayındaki örnek noktalar.
- `labels`: Her noktanın sınıf etiketi.
- `Voronoi(points)`: Uzayı en yakın noktaya göre bölümlere ayırır.
- `voronoi_plot_2d`: Bu bölünmeyi görselleştirir.

Ortaya çıkan diyagramda her hücre bir veri noktasına aittir ve hücre içindeki tüm noktalar o veri noktasına en yakındır. Hücre sınırları 1-NN karar sınırlarını temsil eder.

Sonuç

Bu örnek, 1-NN yönteminin geometrik karşılığını açıkça gösterir. Voronoi diyagramı sayesinde, sınıflandırmanın aslında uzayın en yakın komşuya göre parçalanması olduğu doğrudan gözlemlenebilir.

6 k-NN'de k Nasıl Seçilir?

k büyüdükçe teoriye göre daha iyi sonuçlar elde edilir, ancak pratikte önemli sınırlamalar vardır.

6.1 Teorik Bakış

Teoride, veri sayısı sonsuzsa k ne kadar büyükse hata oranı o kadar düşer ve Bayes hatasına yaklaşır. Ancak burada önemli bir şart vardır: seçilen k komşunun gerçekten x'e yakın olması gerekir. Bu ise ancak sonsuz veri durumunda mümkündür.

Pratikte veri sınırlı olduğu için büyük k seçildiğinde uzak noktalar da komşu olarak dahil edilir ve bu durum hataya yol açar.

Önemli

Teoride büyük k iyidir, ancak pratikte çok büyük k kötü olabilir. Doğru k, veri miktarına ve dağılıma bağlıdır.

6.2 Pratikte k Seçimi: Denge Problemi

Pratikte k seçimi iki karşıt gereksinim arasında bir denge problemidir:

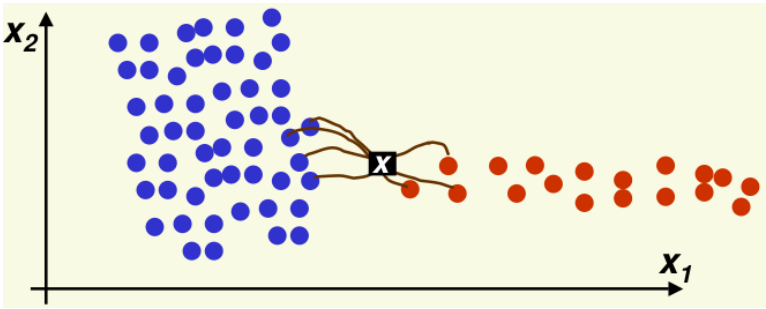
k yeterince büyük olmalıdır. Hata oranını düşürmek için belirli bir büyüklükte k gereklidir. Çok küçük k seçildiğinde karar sınırları gürültülü olur ve model aşırı hassas davranır.

k aynı zamanda küçük olmalıdır. Sadece yakın komşuların dikkate alınması istenir. Çok büyük k seçildiğinde uzak noktalar da dahil olur ve karar sınırları fazla düzleşir.

Bu iki durum arasında denge kurmak kolay değildir. Doğru k, veri yapısına bağlıdır ve dikkatli seçilmelidir.

6.3 k Seçiminin Etkisi: Somut Örnek

Aşağıdaki örnek, k değerinin sınıflandırmayı nasıl doğrudan değiştirdiğini göstermektedir.



Şekil 5: k seçiminin sınıflandırmaya etkisi. Mavi noktalar bir sınıfı, kırmızı noktalar diğer sınıfı temsil etmektedir. Ortadaki siyah x sınıfı bilinmeyen noktadır. Çizilen bağlantılar x'in en yakın komşularını göstermektedir. x iki kümenin sınırına yakın bir konumdadır.

Küçük k durumu. $k = 1, \ldots, 7$ için en yakın komşuların çoğu kırmızıdır. Dolayısıyla x kırmızı sınıfa atanır ve doğru sınıflandırma yapılır. Küçük k, yerel bilgiye dayanır ve sınır bölgelerinde daha doğru karar verebilir.

Büyük k durumu. k büyüdükçe daha uzak mavi noktalar da komşu olarak dahil edilir. Bu durumda mavi noktalar çoğunluk haline gelir ve x yanlış şekilde mavi sınıfa atanır. Büyük k, global ortalamaya kayar ve sınır noktalarında hata yapar.

Sonuç

k çok büyütülürse, model yakın komşular yerine uzak ve alakasız noktaların etkisine girer ve hatalı karar verir. Küçük k yerel bilgiye dayanarak daha doğru karar verebilirken, büyük k global ortalamaya kayarak sınır bölgelerinde hataya yol açar. Doğru k bu iki uç arasında dengelenmelidir.

7 k-NN: Hesaplama Maliyeti

k-NN'in en büyük dezavantajı, hesaplama maliyetinin yüksek olmasıdır.

7.1 Maliyet Nasıl Hesaplanıyor?

Varsayalım ki n veri sayısını, d boyut sayısını ve k komşu sayısını göstersin. Adım adım maliyet şöyle oluşur:

- Bir nokta ile mesafe hesaplamak: $O(d)$
- Tüm veriyle karşılaştırmak: $O(nd)$
- En yakın k komşuyu bulmak: $O(knd)$

Toplam hesaplama karmaşıklığı:

$$O(knd) \tag{2}$$

Bu ifade şu anlama gelir: her yeni veri noktası için tüm veri setine bakmak zorunludur. Veri büyüdükçe hesaplama çok pahalı hale gelir.

7.2 Temel Çelişki

k-NN'in iyi çalışması için çok veri gerekir; ancak veri arttıkça hesaplama maliyeti de hızla artar. Bu durum yöntemin temel çelişkisini oluşturur.

Sonuç

k-NN basit ve etkili bir yöntemdir, ancak büyük veri ve yüksek boyutta yavaş ve maliyetlidir. Kısaca: daha fazla veri daha iyi doğruluk sağlar, ama çok daha yüksek hesaplama maliyetine yol açar.

8 k-NN'de Karmaşıklığı Azaltma: Arama Ağacı ve Prototipler

Buradaki yaklaşım, veriden bazı örnekleri seçmekten ziyade, veriyi hiyerarşik bir arama yapısı (search tree) içinde temsil etmektir. Amaç, tüm veri üzerinde tek tek arama yapmak yerine, benzerliklere göre organize edilmiş bir yapı üzerinden daha hızlı arama yapmaktır.

8.1 Yöntemin Çalışma Prensibi

Bu yöntemde veri noktaları benzerliklerine göre gruplanır ve bu gruplar bir ağaç yapısı içinde temsil edilir. Her düğüm, altındaki veri noktalarını temsil eden bir "prototip" gibi davranır.

Arama sürecinde tüm veri yerine önce ağacın üst seviyelerinde karar verilir, daha sonra ilgili alt dallara inilir. Böylece gereksiz karşılaştırmalar yapılmaz.

8.2 Avantaj

Tüm veri üzerinde arama yapmak yerine daha az sayıda karşılaştırma yapılır ve hesaplama maliyeti düşer.

8.3 Dezavantajlar

- İyi bir ağaç yapısı kurmak zor bir problemdir.
- Arama sırasında bazı dallar elenebilir. Bu nedenle gerçek en yakın komşu her zaman bulunamayabilir ve karar sınırları değişebilir.

Sonuç

Bu yaklaşım, k-NN'i hızlandırmak için veriyi organize eder. Ancak hız kazanımı karşılığında, en yakın komşuyu garanti etme özelliği zayıflayabilir.

9 k-NN'de Mesafe Seçimi ve Özellik Ağırlıklandırma

k-NN yönteminde en kritik adım, iki nokta arasındaki "yakınlığın" nasıl ölçüldüğüdür. Genellikle bu amaçla Öklid mesafesi (Euclidean distance) kullanılır:

$$D(a,b) = \sqrt{\sum_k (a_k - b_k)^2} \tag{3}$$

Bu yaklaşımda her özellik (feature) eşit derecede önemli kabul edilir. Ancak pratikte bu varsayım çoğu zaman doğru değildir.

9.1 Problem

Bazı özellikler sınıfları ayırmada çok etkilidir, bazıları ise neredeyse hiçbir bilgi taşımaz. Öklid mesafesi tüm özelliklere eşit ağırlık verdiğinden, önemli özelliklerin etkisi azalabilir.

9.2 Çözüm: Özellik Ağırlıklandırma

Bu problemi çözmek için her özelliğe bir ağırlık verilir:

$$D(a,b) = \sqrt{\sum_k w_k \, (a_k - b_k)^2} \tag{4}$$

Burada w_k, k. özelliğin önemini belirler. Büyük w_k değeri daha önemli bir özelliği, küçük w_k değeri ise daha az önemli bir özelliği ifade eder.

9.3 Ağırlıklar Nasıl Belirlenir?

Ağırlıklar eğitim verisi kullanılarak öğrenilebilir. Amaç, sınıflandırma performansını artırmaktır. Bu süreçte ağırlıklar artırılır veya azaltılır ve en iyi sonuç veren değerler seçilir.

Sonuç

Standart k-NN tüm özellikleri eşit görürken, ağırlıklı k-NN önemli özellikleri öne çıkarır. Doğru mesafe tanımı, k-NN performansını doğrudan belirler.

10 k-NN'de Özellik Ölçekleme (Normalization)

k-NN'de mesafe hesabı doğrudan özellik değerlerine bağlıdır. Eğer özellikler farklı ölçeklerdeyse, mesafe hesabı ciddi şekilde bozulur.

10.1 Problem

Örneğin bir özellik 1–2 arasında değerler alırken, diğer özellik 100–200 arasında değerler alıyorsa, mesafeyi neredeyse tamamen büyük ölçekli özellik belirler. Küçük ölçekli özelliğin etkisi yok denecek kadar azalır. Yani model yanlış öncelik verir.

10.2 Çözüm: Normalize Etme

Her özelliği aynı ölçeğe getirmek gerekir. Bunun için standart normalizasyon (z-score normalization) uygulanır:

$$x' = \frac{x - \mu}{\sigma} \tag{5}$$

Burada μ ortalamayı, σ standart sapmayı gösterir. Bu dönüşüm sonucunda her özelliğin ortalaması 0, varyansı 1 olur.

10.3 Nasıl Uygulanır?

Her özellik için ayrı ayrı şu adımlar izlenir:

1. Ortalama (μ) hesaplanır.
2. Varyans (σ^2) hesaplanır.
3. Tüm değerler $x' = \frac{x-\mu}{\sigma}$ formülü ile dönüştürülür.

Bu dönüşüm sayesinde tüm özellikler eşit ölçeğe gelir, mesafe hesabı daha dengeli olur ve model doğru özellikleri dikkate alır.

Sonuç

Normalize edilmemiş veri yanıltıcı mesafe hesabına yol açarken, normalize edilmiş veri daha doğru ve güvenilir k-NN sonuçları üretir. Ölçekleme yapılmadan k-NN kullanmak ciddi hatalara yol açabilir.

11 k-NN Özeti

k-NN basit ama güçlü bir yöntemdir.

11.1 Avantajlar

- Herhangi bir dağılım varsayımı gerektirmez.
- Uygulaması kolay ve sezgiseldir.
- Yeterli veri varsa iyi sınıflandırma yapar.

11.2 Dezavantajlar

- Doğru k seçimi zordur.
- Hesaplama maliyeti yüksektir.
- Yüksek doğruluk için çok veri gerekir; bu ihtiyaç, parametrik varsayım olmadan tam olarak giderilemez.

Sonuç

k-NN basit ve esnek bir yöntemdir, ancak veri ve hesaplama maliyeti açısından pahalıdır.